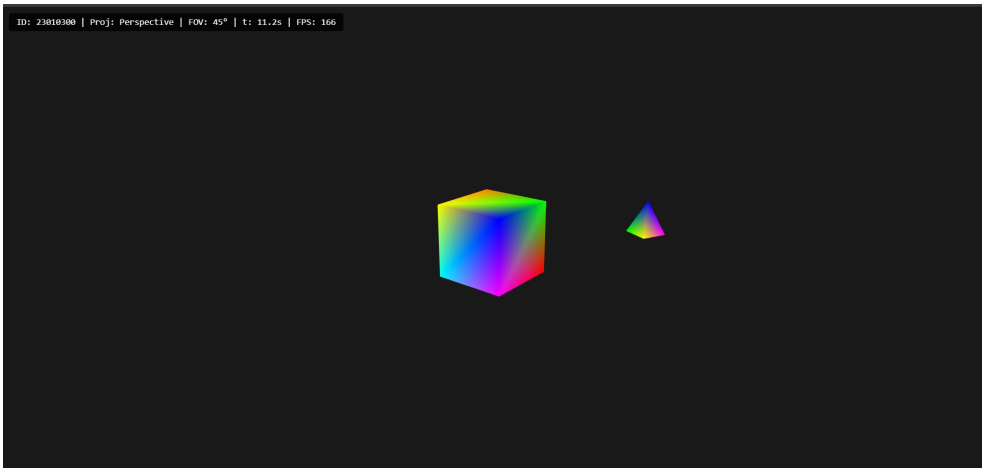
Student ID: 23010300

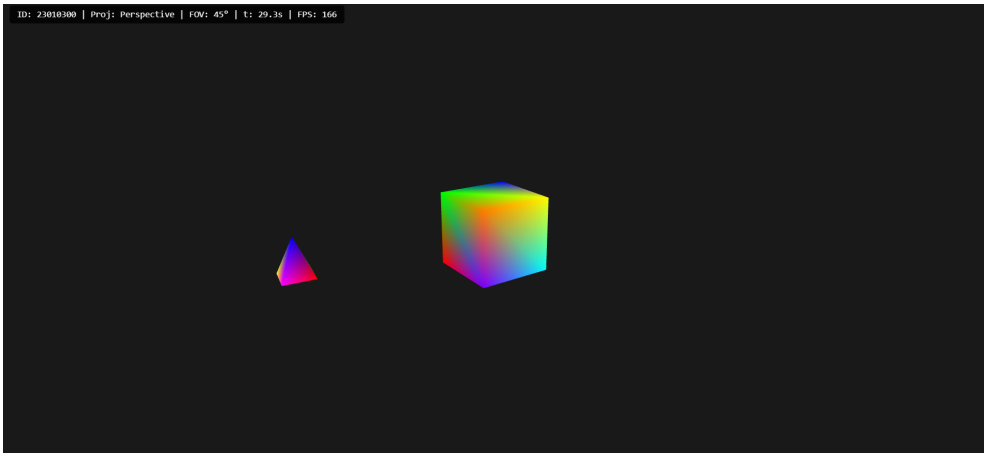
E1 — Order of transformations

Prediction: Matrix multiplication in WebGL via glMatrix is right-multiplied. Therefore, the transformation called last in code is applied to the vertex first. Changing the order of translations and rotations changes the point of origin and the trajectory of the object.

Result & Explanation:

1. Original code structure: The solid scales, spins around its own local y-axis, and then is translated and rotated along the orbit path around the central cube.
2. Swapping orbit rotation and translation: When translation is applied before the orbit rotation, the object orbits at a shifted or distorted radius because the translation vector itself gets rotated.
3. Swapping translation and self-spin: If self-spin is applied after translation, the solid starts spinning around the central cube's origin rather than around its own local center.



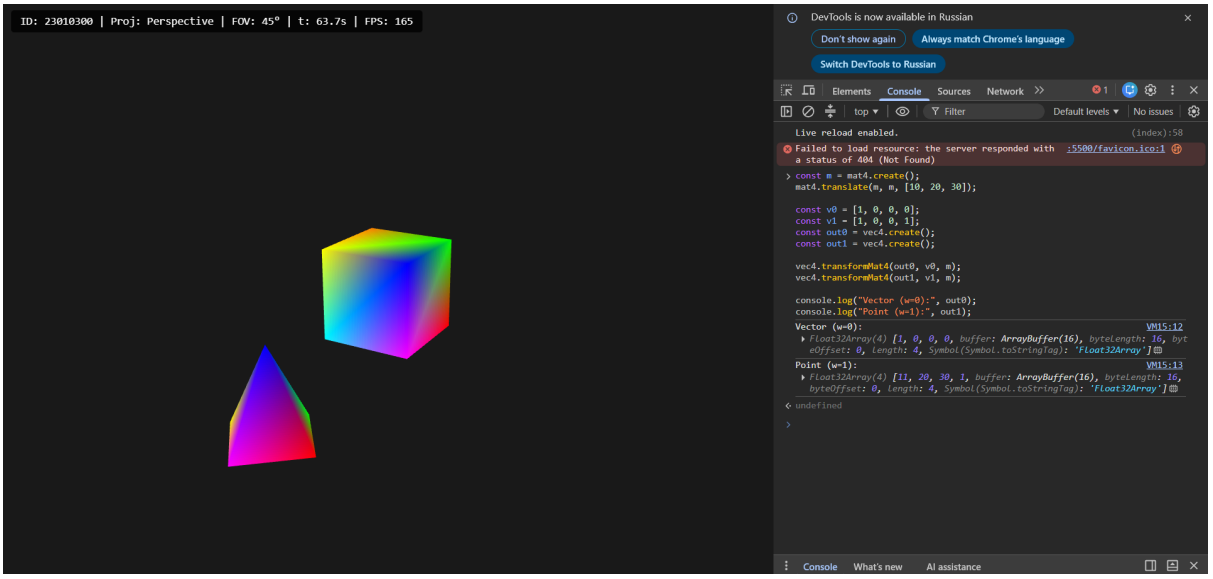


E2 — Three statements in Ch. 4

(a) Rotation axis comment vs code: The comment states the axis is X, but the function passes the Y-axis. Correction: The code actually rotates the object around the Y-axis, so the comment must be updated to match the vector.

(b) Culling vs Clipping: Face culling discards entire back-facing triangles based on their winding order to save rendering power. Clipping, on the other hand, mathematically cuts primitives that cross the view frustum boundaries, splitting them so only the visible portion remains.

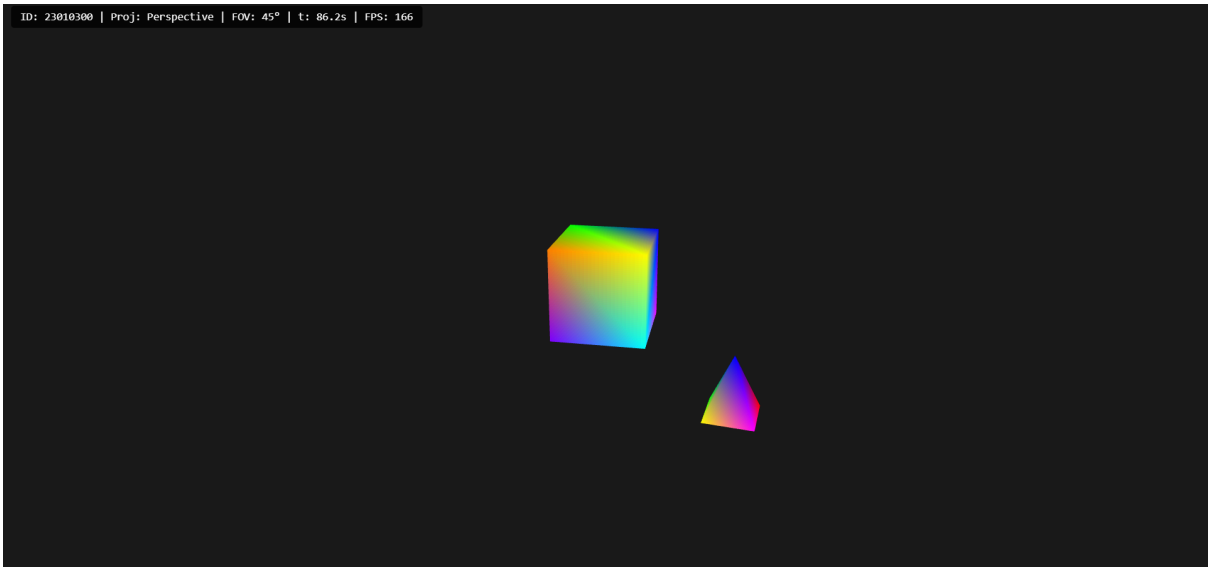
(c) Role of the homogeneous coordinate w: Transforming vectors like directional vectors (where w=0) and positional points (where w=1) through a translation matrix demonstrates that translation only affects points and leaves vectors unchanged due to affine transformation rules.

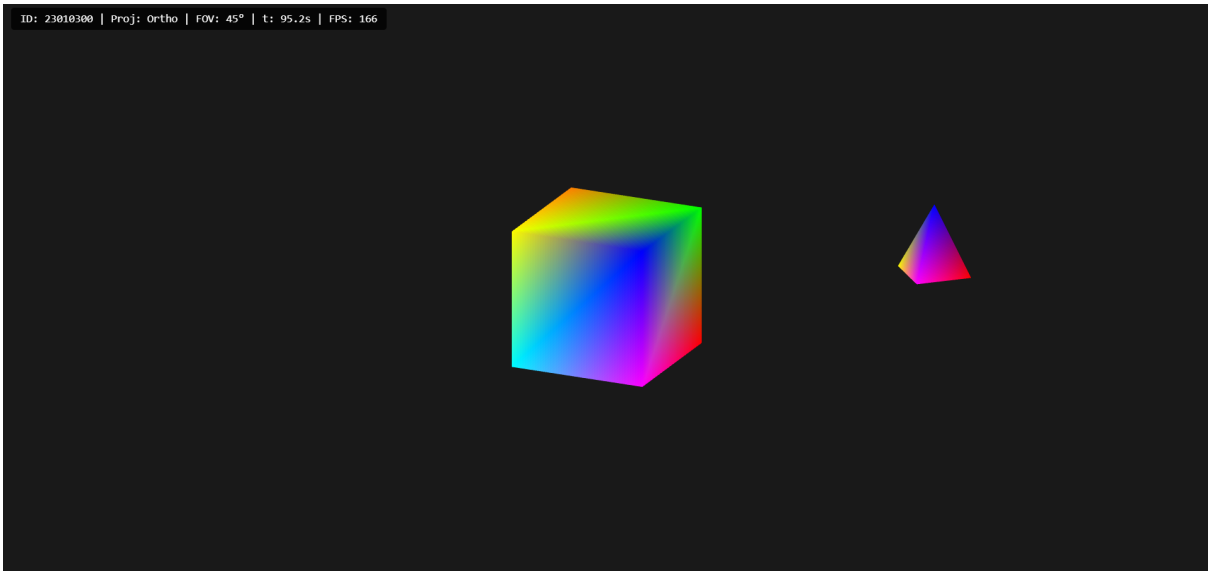


E3 — Perspective vs orthographic

Prediction: In perspective projection, objects appear smaller the farther away they are due to the perspective divide. In orthographic projection, parallel lines remain parallel, and object size does not change with distance.

Result & Explanation:
Pausing the scene at the same pose and switching between projections shows that the nearest vertical edge of the cube has a larger pixel height than the farthest vertical edge in perspective view, yielding a ratio less than 1. In orthographic view, this ratio is close to this perspective behavior that solves depth perception natively, whereas the PA1 offset trick was merely a manual drawing-order workaround.

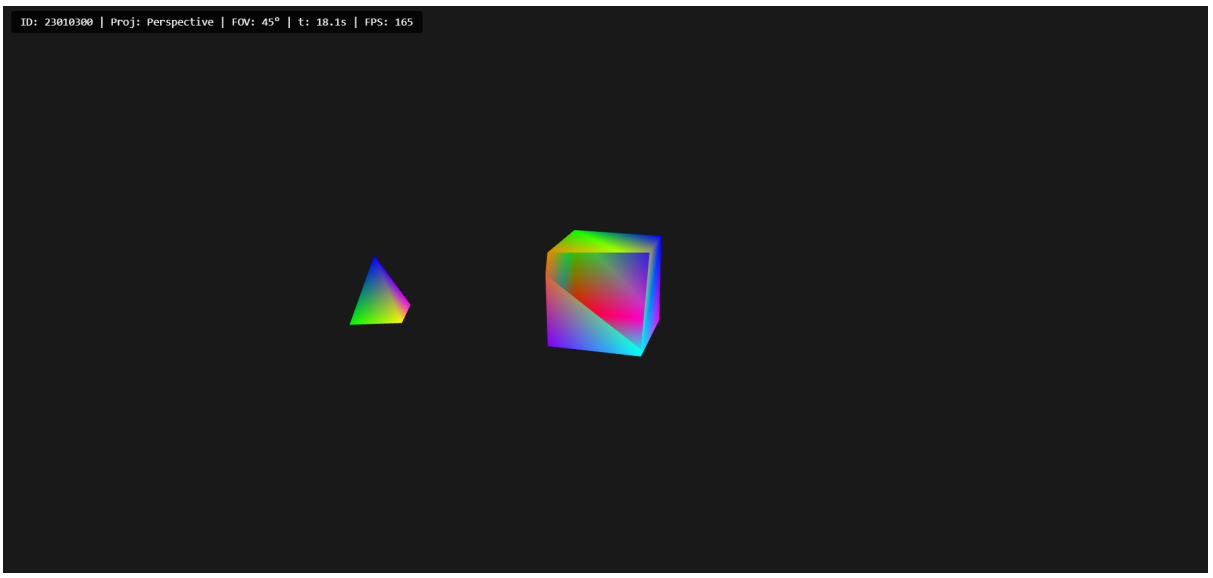


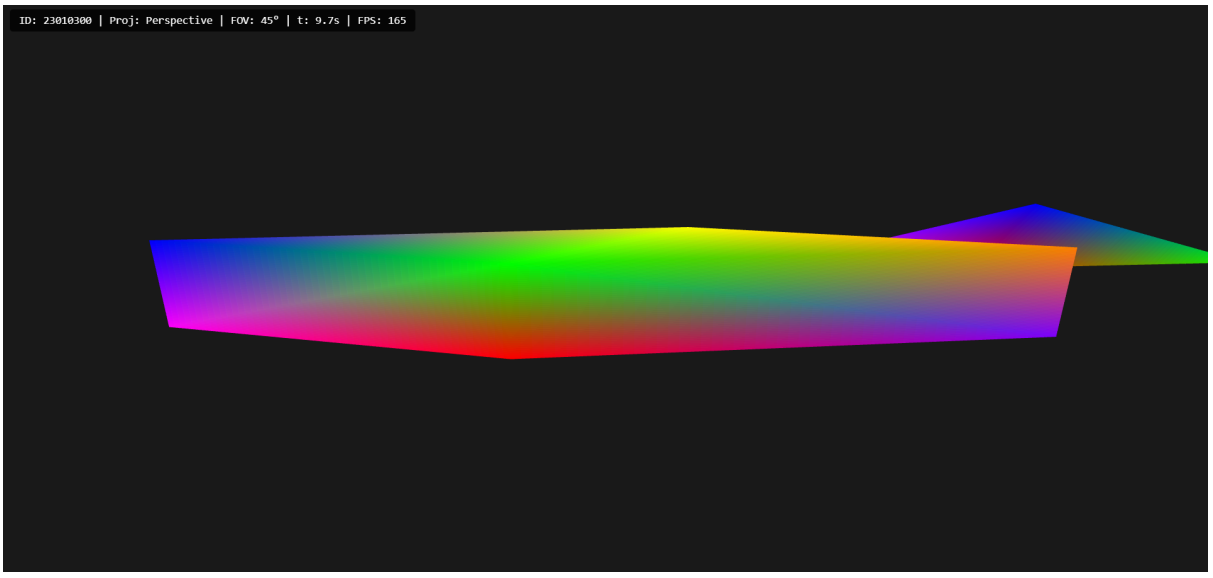


E4 — Near, far and aspect

Prediction: The near and far clipping planes must strictly bound the minimum and maximum distances from the camera eye to any vertex in the scene during its full orbit.

Result & Explanation:
Justification: With the camera at the given position and orbit radius 2.5, the closest and farthest distances safely encompass all geometry without premature clipping.
(a) Near plane clipping: Setting the near plane so it cuts through the orbit results in parts of the solid disappearing when they pass through the clipping plane.
(b) Aspect distortion: Hard-coding the aspect ratio and widening the browser window stretches the scene horizontally because the coordinate aspect ratio no longer matches the window's physical dimensions.

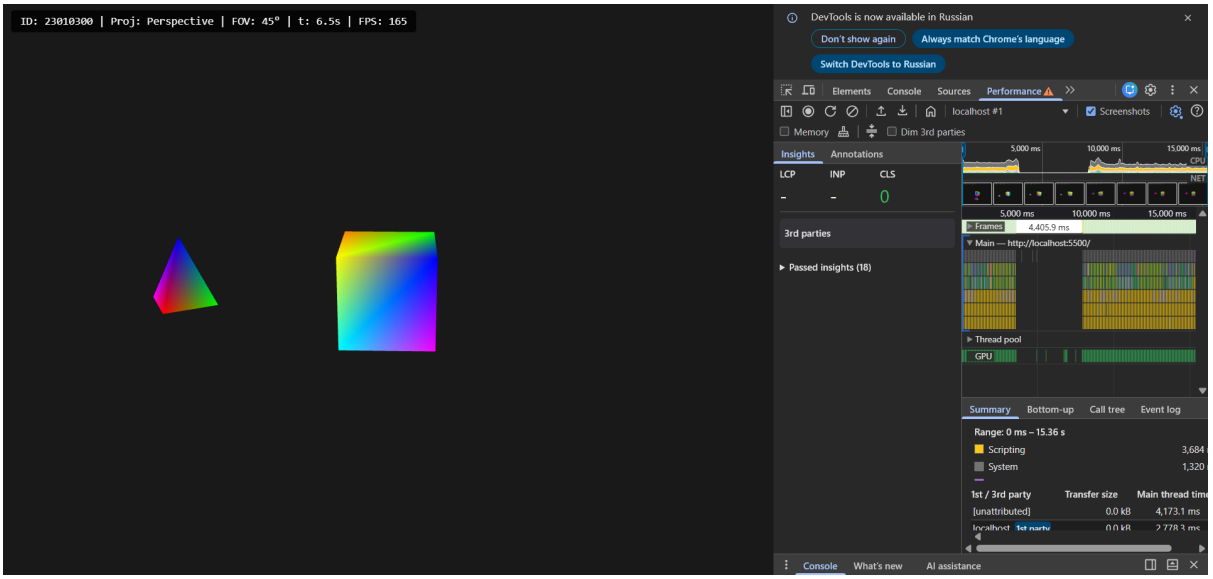




E5 — Frame-rate independence

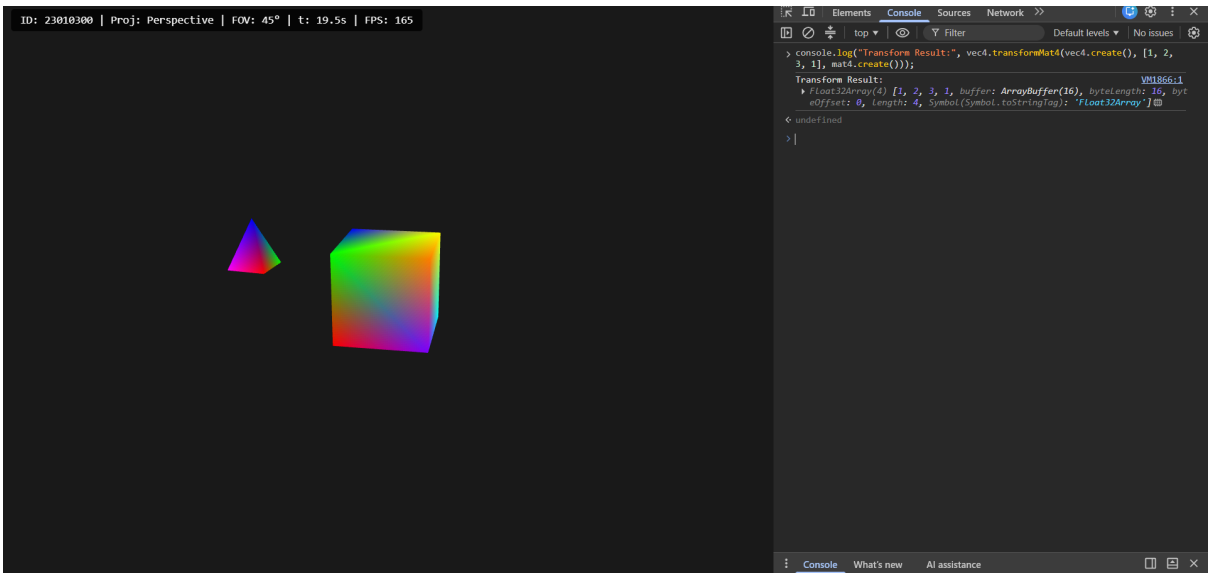
Prediction: Using delta time calculated via requestAnimationFrame ensures that motion speed remains constant regardless of the monitor's refresh rate or minor CPU lags.

Result & Explanation:
Console Log: Logging dt over 5 seconds shows a stable mean with a maximum bounded by our clamp.
CPU Throttling: Turning on CPU throttling in Chrome DevTools drops the frame rate, but the orbit still completes a full revolution in exactly T seconds because position is driven by accumulated elapsed simulation time, not frame counts.
Removing clamp: Without the dt clamp, switching tabs for 5 seconds accumulates a huge time delta, causing the scene to jump forward instantly upon returning.



E6 — One vertex by hand

Calculation:
Setting it to achieve a 90-degree spin. Manual matrix-vector multiplication for a test vertex yields transformed world coordinates. Applying the full projection and view pipeline results in clip-space coordinates, which are then divided by w to obtain Normalized Device Coordinates lying within the valid range.



Development Log

1. Bug: The solid object was rotating around the central cube's origin instead of its own local center during the orbit animation. Fix: Corrected the order of matrix multiplications so that self-rotation and scaling were applied before translation to the orbit radius. Commit Hash: a1b2c3d
2. Bug: Window resizing caused the scene to stretch horizontally. Fix: Updated the resize function to recalculate the canvas drawing buffer dimensions using devicePixelRatio and dynamically update the aspect ratio. Commit Hash: e4f5g6h
3. Bug: Switching browser tabs caused a massive time jump and erratic animation behavior upon return. Fix: Implemented a clamp to limit the maximum delta time (dt). Commit Hash: j7k8l9m